

PaladinShield — Guía técnica para pitch (3 min) y preguntas difíciles

Documento de estudio · PaladinShield (ClearSign AI) · Runtime Enforcement Layer (REL) para Solana
Versión: referencia al código MV3 actual · Motor semántico: OpenAI **gpt-4o-mini** (JSON mode)

Resumen ejecutivo (memorizar)

PaladinShield es una **capa de ejecución en tiempo de ejecución**: intercepta las llamadas de firma del wallet en la página (`signTransaction` , `signAllTransactions` , `signMessage`), **no devuelve la Promesa** hasta que un **auditor semántico** (LLM con rol fijo) emite un veredicto estructurado y el usuario **confía o bloquea**. No es solo un asistente que explica: **hace cumplir default-deny** y puede **persistir evidencia forense con hash SHA-256**. La demo es una **extensión de navegador**; la visión es **integrar esta REL en wallets de Solana** para protección universal en el punto de firma.

1. Tecnologías aplicadas

Área	Tecnología	Para qué sirve
Extensión	Chrome Manifest V3	Service worker como runtime principal; popup como UI de veredicto
Inyección	Script en página + <code>web_accessible_resources</code>	<code>inject.js</code> corre en el contexto de la página y envuelve <code>window.solana</code>
Aislamiento	Content script + <code>postMessage</code>	El mundo de la página no puede llamar APIs de extensión directamente; el puente es mensajes
Mensajería	<code>chrome.runtime.sendMessage</code> , listeners	Content script → background; popup sincroniza estado
Persistencia	<code>chrome.storage.local</code>	Estado REL, historial de amenazas, reportes forenses
Red	Fetch a OpenAI Chat Completions	Motor semántico con <code>response_format: json_object</code>
IA	OpenAI gpt-4o-mini	Veredicto riesgo / accion / mensaje (no chat libre)
Resiliencia	AbortController ~4 s + fail-safe local	Si la API falla o hace timeout → Alto + Bloquear por defecto
Heurísticas	Código en <code>translator.js</code>	<code>signMessage</code> (patrones phishing/urgencia); honey-pot en instrucciones antes de llamar al LLM
Criptografía	SHA-256 (Web Crypto en el flujo forense)	<code>paladinForensicHash</code> sobre JSON canónico del paquete de integridad
UI	HTML/CSS/JS del popup + Evidence Hub	Veredicto, botones CONFIAR/BLOQUEAR, exportación de certificado

2. Funcionamiento (flujo paso a paso)

- El usuario abre un dApp. El proveedor expone `window.solana` (Phantom, Solflare, etc.).

- 2. `inject.js` reemplaza los métodos de firma por versiones envueltas. Si detecta que ya están envueltos, no duplica.
- 3. Al llamar por ejemplo `signTransaction` , el wrapper:
 - genera un `requestId` ;
 - construye un `payload` (transacciones serializadas, origen, metadatos para análisis);
 - crea `decisionPromise` = esperar decisión explícita del service worker;
 - publica el payload vía `postMessage` al content script.
- 4. `content_script.js` reenvía el mensaje al background como `SIGNATURE_INTENT` o `MESSAGE_SIGNATURE_INTENT` (para `signMessage`).
- 5. `background.js` :
 - registra la petición pendiente (`pendingSignatureRequests`);
 - guarda estado `analyzing` y abre el `popup` de veredicto;
 - llama `translateTransaction` → puede usar heurísticas rápidas o **OpenAI** con prompt de auditor;
 - guarda `completed` con `analysisResult` ;
 - si **Alto** o **Bloquear**, puede crear **reporte forense** y persistirlo.
- 6. El `popup` muestra riesgo, mensaje y acciones. El usuario pulsa **CONFIAR** o **BLOQUEAR** (o cierra ventana → política **default-deny** según handlers).
- 7. El background envía la decisión al content script → `inject.js` recibe `SIGNATURE_DECISION` → `decisionPromise` se **resuelve** (aprobar) o **rechaza** (bloquear).
- 8. Solo si se aprueba, el wrapper ejecuta `original.apply` y la firma llega al wallet real.

Idea clave: mientras `await decisionPromise` no termine, **el JavaScript del dApp sigue esperando**; la firma **no ocurre en paralelo** “por detrás”.

3. Diferenciación del producto

Enfoque típico	PaladinShield
Simular la tx y avisar	Retiene la Promesa hasta veredicto y decisión humana
Explicar en lenguaje natural (copiloto)	Emite JSON de política que alimenta bloqueo y UI, no chat abierto
Confiar en “el sitio parece bueno”	El prompt prohíbe bajar el riesgo solo por reputación del origen vs. análisis del payload
Solo <code>signTransaction</code>	Paridad con <code>signMessage</code> (phishing de sesión / SIWE-like)
Sin prueba después del hecho	Evidence Hub : certificado + hash verificable sobre el bundle de incidente
Producto = reemplazar wallet	Tesis middleware / REL : hoy extensión; mañana integrado en wallets del ecosistema

Memorizar en una frase: “No estamos en la categoría ‘asistente que explica’, estamos en ‘infraestructura que ejecuta default-deny en el boundary de firma.”

4. Esquema de pitch (~3 minutos)

Tiempo	Qué decir
--------	-----------

0:00– 0:30	Problema: un solo click ciego o un <code>signMessage</code> de “verificación” puede vaciar fondos. Las advertencias solas no frenan la ejecución.
0:30– 1:15	Qué es PaladinShield: REL que envuelve <code>window.solana</code> , promise-gating — la firma no continúa hasta auditoría semántica + decisión explícita. Default-deny si se cierra el popup.
1:15– 1:45	Cómo: extensión MV3, inyección en página, service worker orquesta OpenAI gpt-4o-mini en modo JSON + heurísticas + fail-safe 4 s.
1:45– 2:15	Por qué no somos ChainGPT/GuardSQL “puro”: ellos educan o simulan; nosotros bloqueamos físicamente la resolución de la Promesa y dejamos evidencia con hash .
2:15– 2:45	Roadmap: hoy demo en extensión; post-hackathon REL nativo en wallets Solana para que no dependa cada usuario de instalar un add-on.
2:45– 3:00	Cierre: “ <i>PaladinShield convierte la firma en un punto de control de política verificable, no en un gesto irreversible sin contexto.</i> ”

5. Preguntas duras y respuestas sugeridas

P: “¿No es solo otra extensión de seguridad?”

R: La diferencia está en **dónde** actuamos: en la **Promesa de firma del proveedor**. Muchas herramientas muestran riesgo pero la llamada puede seguir; nosotros **no resolvemos** esa Promesa hasta política + usuario. Eso es enforcement en el camino de ejecución, no solo UX.

P: “¿El LLM puede equivocarse?”

R: Tres mitigaciones: (1) **rol de auditor** con salida **JSON validada** (`riesgo` , `accion` , `mensaje`); (2) **heurísticas** antes del modelo en algunos caminos (`signMessage` , honey-pot); (3) **fail-safe**: timeout o error de API → **Alto + Bloquear**. Preferimos falsos positivos que falsos negativos en demo agresiva; el usuario puede **confiar** tras revisión si la política lo permite.

P: “¿Un sitio malicioso puede evitar PaladinShield?”

R: Un dApp **no puede** quitar nuestra extensión. Pueden intentar **no usar** el adaptador estándar, iframe aislados, o flujos fuera del navegador; por eso el roadmap es **wallet nativo** y capas de red (RPC Guard). Hoy cubrimos el **caso más común**: apps web que firman vía `window.solana` .

P: “¿Por qué confiar en OpenAI para seguridad?”

R: OpenAI es el **motor de juicio semántico** bajo prompt estricto, no la única línea de defensa. En producción el plan es **backend proxy** sin claves en el cliente (`SECURITY_ROADMAP.md`). El producto es **REL + política + trazabilidad**, no “OpenAI dice confiar”.

P: “¿Qué pasa si el usuario cierra el popup?”

R: Implementación orientada a **default-deny**: cerrar sin aprobar debe **bloquear** la firma pendiente (rechazo de Promesa). Eso es coherente con “mejor no firmar que firmar a ciegas”.

P: “¿Cómo verifican la evidencia forense?”

R: Se calcula `paladinForensicHash` como **SHA-256** sobre un JSON **canónico** que incluye el certificado y campos ligados al incidente. Si alguien altera texto o campos, **el hash no coincide**. Eso permite interoperar con custodios o

reportes externos.

P: “¿Compiten con Phantom o con GuardSQL?”

R: No reemplazamos wallet; **interceptamos la interfaz estándar** que las wallets exponen en la web. GuardSQL en el corpus enfatiza **simulación**; nosotros enfatizamos **promise gate + política semántica + forensics**. ChainGPT **explica**; nosotros **enforceamos** con JSON de veredicto.

P: “¿Es seguro tener la API key en el código?”

R: No para producción ni repo público. En hackathon/demo se aceptó fricción cero; `SECURITY_ROADMAP.md` documenta rotación y **backend seguro**. En pitch: transparencia total y plan claro.

P: “¿Y la privacidad del usuario?”

R: El intent se envía al proveedor LLM para análisis; en producción habría **minimización de datos**, retención corta, posible dedi-cated stack, y opción de **modelo on-prem / región**. Aclararlo en roadmap.

P: “¿Por qué Solana?”

R: Alta densidad de firmas en DeFi/juegos/apps web; mismo patrón `window.solana` en muchos productos; REL aquí protege **antes** de que los errores humanos lleguen on-chain.

6. Frases cortas para cerrar respuestas

- «**La simulación informa; el REL retiene la ejecución.**»
- «**El asistente explica; la infraestructura bloquea.**»
- «**El veredicto es dato estructurado, no conversación.**»
- «**La evidencia ata la narrativa al hash.**»

7. Exportar este documento a PDF

En **Cursor**: vista previa Markdown → extensión **Markdown PDF** → Export PDF.

En **terminal** (desde la raíz del repo):

```
npx --yes md-to-pdf "docs/PALADINSHIELD_GUIA_PITCH_Y_QA_ESP.md"
```

Se generará `PALADINSHIELD_GUIA_PITCH_Y_QA_ESP.pdf` junto al `.md` (la primera ejecución puede tardar por Chromium).

Última revisión alineada con manifest MV3, OpenAI gpt-4o-mini y roadmap REL en wallets.